model

April 18, 2026

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import shap
from sklearn.utils import shuffle
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from matplotlib.patches import Patch
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from sklearn.metrics import (
    roc_auc_score, accuracy_score, roc_curve, brier_score_loss,
    precision_score, f1_score, confusion_matrix, classification_report
)
from sklearn.model_selection import train_test_split
from skopt import BayesSearchCV
from skopt.space import Real, Integer, Categorical


#   Helpers

def style_plot(ax, title, xlabel, ylabel, legend=True, grid=True):
    ax.set_title(title, fontsize=12, fontweight='bold', pad=15, color='black')
    ax.set_xlabel(xlabel, fontsize=12, labelpad=10, color='black')
    ax.set_ylabel(ylabel, fontsize=12, labelpad=10, color='black')
    if grid:
        ax.grid(True, axis='y', alpha=0.3, linestyle='-', linewidth=0.5,␣
 ↪color='#666666')
        ax.grid(False, axis='x')
    if legend:
        handles, labels = ax.get_legend_handles_labels()
        if handles:
```

```
            ax.legend(fontsize=10, frameon=True, loc='upper left',␣
 ↪bbox_to_anchor=(1, 1))
    ax.tick_params(axis='both', which='major', labelsize=10, colors='black')
    ax.set_facecolor('#f5f5f5')
    for spine in ax.spines.values():
        spine.set_color('#cccccc')
    return ax

def find_best_threshold(y_true, y_prob):
    fpr, tpr, thresholds = roc_curve(y_true, y_prob)
    return thresholds[np.argmax(tpr - fpr)]

def compute_sensitivity(y_true, y_prob, threshold):
    y_pred = (y_prob >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return tp / (tp + fn) if (tp + fn) > 0 else 0.0

def evaluate_model(y_true, y_prob, threshold, name):
    y_pred = (y_prob >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0
    acc   = accuracy_score(y_true, y_pred)
    prec  = precision_score(y_true, y_pred)
    f1    = f1_score(y_true, y_pred)
    auc   = roc_auc_score(y_true, y_prob)
    brier = brier_score_loss(y_true, y_prob)
    print(f"\n{name} (threshold={threshold:.3f})")
    print(f"  TN={tn}  FP={fp}  FN={fn}  TP={tp}")
    print(f"  Sensitivity: {sensitivity*100:.2f}%  Specificity:␣
 ↪{specificity*100:.2f}%")
    print(f"  F1: {f1:.4f}  Accuracy: {acc*100:.2f}%  Precision: {prec*100:.
 ↪2f}%")
    print(f"  ROC-AUC: {auc:.4f}  Brier: {brier:.4f}")
    print(classification_report(y_true, y_pred, digits=4))
    return confusion_matrix(y_true, y_pred), {
        'sensitivity': sensitivity, 'specificity': specificity,
        'f1_score': f1, 'precision': prec, 'accuracy': acc, 'roc_auc': auc
    }

def categorize_risk(prob, low=0.3, high=0.7):
    if prob < low:  return "Low Risk"
    if prob < high: return "Moderate Risk"
    return "High Risk"

def get_top_shap_features(shap_vals_row, feat_names, n_top=5, positive=True):
    shap_vals_row = np.array(shap_vals_row, dtype=float).flatten()
```

```
    indices = np.argsort(shap_vals_row)[-n_top:][::-1] if positive else np.
 ↪argsort(shap_vals_row)[:n_top]
    features = []
    for idx in indices:
        idx = int(idx)
        if positive and shap_vals_row[idx] > 0:
            features.append(f"{feat_names[idx]} (+{shap_vals_row[idx]:.3f})")
        elif not positive and shap_vals_row[idx] < 0:
            features.append(f"{feat_names[idx]} ({shap_vals_row[idx]:.3f})")
    while len(features) < n_top:
        features.append("")
    return features


#   Data

train_data = pd.read_csv("training_dataset.csv")
X_full = train_data.drop(columns=["Diagnosed", "Anxiety Level (1-10)"])
y_full = train_data["Diagnosed"]
X_full, y_full = shuffle(X_full, y_full, random_state=42)

X_train, X_val, y_train, y_val = train_test_split(
    X_full, y_full, test_size=0.2, random_state=42, stratify=y_full
)

test_data = pd.read_csv("testing_dataset.csv")
X_test = test_data.drop(columns=["Diagnosed", "Anxiety Level (1-10)"])
y_test = test_data["Diagnosed"]

numeric_features     = X_train.select_dtypes(include=[float, int]).columns.
 ↪tolist()
categorical_features = X_train.select_dtypes(include=["object", "string"]).
 ↪columns.tolist()

preprocessor = ColumnTransformer([
    ("num", MinMaxScaler(), numeric_features),
    ("cat", OneHotEncoder(drop="first", sparse_output=False),␣
 ↪categorical_features)
])


#   Search Spaces

logreg_space = {
    'classifier__C':        Real(0.01, 10, prior='log-uniform'),
    'classifier__l1_ratio': Categorical([0, 0.5, 1]),
    'classifier__solver':   Categorical(['saga']),
```

```
}

rf_space = {
    'classifier__n_estimators':      Integer(50, 200),
    'classifier__max_depth':         Integer(3, 15),
    'classifier__min_samples_split': Integer(2, 20),
    'classifier__min_samples_leaf':  Integer(1, 20),
    'classifier__max_features':      Categorical(['sqrt', 'log2']),
}

xgb_space = {
    'classifier__n_estimators':     Integer(100, 400),
    'classifier__max_depth':        Integer(3, 10),
    'classifier__learning_rate':    Real(0.05, 0.35, prior='log-uniform'),
    'classifier__subsample':        Real(0.6, 1.0),
    'classifier__colsample_bytree': Real(0.6, 1.0),
    'classifier__min_child_weight': Integer(1, 7),
    'classifier__reg_alpha':        Real(0, 2),
    'classifier__reg_lambda':       Real(0.5, 2.5),
}


#   Pipelines & Tuning

logreg_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42))
])
rf_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(random_state=42, n_jobs=-1))
])
xgb_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", XGBClassifier(eval_metric="logloss", random_state=42,␣
 ↪n_jobs=-1))
])

logreg_search = BayesSearchCV(logreg_pipeline, logreg_space, n_iter=20, cv=3,
                              scoring='roc_auc', n_jobs=-1, random_state=42)
logreg_search.fit(X_train, y_train)

rf_search = BayesSearchCV(rf_pipeline, rf_space, n_iter=20, cv=3,
                          scoring='roc_auc', n_jobs=-1, random_state=42)
rf_search.fit(X_train, y_train)

xgb_search = BayesSearchCV(xgb_pipeline, xgb_space, n_iter=20, cv=3,
```

```
                              scoring='roc_auc', n_jobs=-1, random_state=42)
xgb_search.fit(X_train, y_train)

best_logreg = logreg_search.best_estimator_
best_rf     = rf_search.best_estimator_
best_xgb    = xgb_search.best_estimator_


#   Calibration

scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
best_xgb.named_steps['classifier'].set_params(scale_pos_weight=scale_pos_weight)

logreg_model = CalibratedClassifierCV(best_logreg, method="isotonic", cv=3)
rf_model     = CalibratedClassifierCV(best_rf,     method="isotonic", cv=3)
xgb_model    = CalibratedClassifierCV(best_xgb,    method="isotonic", cv=3)

logreg_model.fit(X_train, y_train)
rf_model.fit(X_train, y_train)
xgb_model.fit(X_train, y_train)


#   Thresholds & Best Model Selection (by validation sensitivity)

logreg_val_prob = logreg_model.predict_proba(X_val)[:, 1]
rf_val_prob     = rf_model.predict_proba(X_val)[:, 1]
xgb_val_prob    = xgb_model.predict_proba(X_val)[:, 1]

logreg_threshold = find_best_threshold(y_val, logreg_val_prob)
rf_threshold     = find_best_threshold(y_val, rf_val_prob)
xgb_threshold    = find_best_threshold(y_val, xgb_val_prob)

val_sensitivity = {
    "Logistic Regression": compute_sensitivity(y_val, logreg_val_prob,␣
 ↪logreg_threshold),
    "Random Forest":       compute_sensitivity(y_val, rf_val_prob,    ␣
 ↪rf_threshold),
    "XGBoost":             compute_sensitivity(y_val, xgb_val_prob,   ␣
 ↪xgb_threshold),
}

best_model_name = max(val_sensitivity, key=val_sensitivity.get)

model_map = {
    "Logistic Regression": (logreg_model, best_logreg, logreg_threshold),
    "Random Forest":       (rf_model,     best_rf,     rf_threshold),
    "XGBoost":             (xgb_model,    best_xgb,    xgb_threshold),
```

```
}
best_calibrated_model, best_raw_pipeline, best_threshold =␣
 ↪model_map[best_model_name]


#   Test Probabilities

logreg_test_prob = logreg_model.predict_proba(X_test)[:, 1]
rf_test_prob     = rf_model.predict_proba(X_test)[:, 1]
xgb_test_prob    = xgb_model.predict_proba(X_test)[:, 1]

df_probs = pd.DataFrame({
    "Patient":                          range(1, 21),
    "Diagnosis (1=Anxiety, 0=No)":      y_test.iloc[:20].values,
    "Logistic Regression (%)":          np.round(logreg_test_prob[:20] * 100,␣
 ↪2),
    "Random Forest (%)":                np.round(rf_test_prob[:20]     * 100,␣
 ↪2),
    "XGBoost (%)":                      np.round(xgb_test_prob[:20]    * 100,␣
 ↪2),
})
print("\nPREDICTED PROBABILITIES - First 20 Patients")
print(df_probs.to_string(index=False))


#   Probability Distribution Plots

def plot_probabilities_horizontal(y_true, y_prob, threshold, name):
    df = pd.DataFrame({"Probability": y_prob, "Diagnosis": y_true})
    fig, ax = plt.subplots(figsize=(10, 6))
    for i, diagnosis in enumerate([0, 1]):
        data  = df[df["Diagnosis"] == diagnosis]
        label = "Non-Diagnosed" if diagnosis == 0 else "Diagnosed"
        ax.hist(data["Probability"], bins=20, alpha=0.6 if diagnosis == 0 else␣
 ↪0.8,
                color=['#a0a0a0', '#404040'][i], label=label,␣
 ↪edgecolor='black', linewidth=0.5)
    ax.axvline(threshold, color='#606060', linestyle="--", linewidth=2,
               label=f"Optimal Threshold = {threshold:.2f}")
    style_plot(ax, f"{name} - Predicted Probability Distribution",
               "Predicted Probability", "Count", grid=False)
    plt.tight_layout()
    plt.show()

plot_probabilities_horizontal(y_test, logreg_test_prob, logreg_threshold,␣
 ↪"Logistic Regression")
```

```
plot_probabilities_horizontal(y_test, rf_test_prob,     rf_threshold,     ␣
 ↪"Random Forest")
plot_probabilities_horizontal(y_test, xgb_test_prob,    xgb_threshold,    ␣
 ↪"XGBoost")


#   Feature Importance

def get_feature_names(X_train, numeric_features, categorical_features):
    temp = ColumnTransformer([
        ("num", MinMaxScaler(), numeric_features),
        ("cat", OneHotEncoder(drop="first", sparse_output=False),␣
 ↪categorical_features)
    ])
    temp.fit(X_train)
    return list(numeric_features) + list(
        temp.named_transformers_["cat"].
 ↪get_feature_names_out(categorical_features)
    )

def plot_feature_importance(pipeline_model, name, feat_names):
    classifier = pipeline_model.named_steps["classifier"]
    fig, ax = plt.subplots(figsize=(8, 6))

    if hasattr(classifier, "feature_importances_"):
        importances = classifier.feature_importances_
        indices = np.argsort(importances)[-15:]

        print(f"\n{name} - Feature Importances (Top 15):")
        print("-" * 50)
        for idx in indices[::-1]:
            print(f"  {feat_names[idx]:<30}: {importances[idx]:.6f}")

        ax.barh(range(len(indices)), importances[indices], color='#404040',
                edgecolor='black', linewidth=0.5)
        ax.set_yticks(range(len(indices)))
        ax.set_yticklabels([feat_names[i] for i in indices], fontsize=10)
        style_plot(ax, f"{name} - Feature Importance (Top 15)",
                   "Feature Importance", "Features", legend=False, grid=False)

    elif hasattr(classifier, "coef_"):
        coefficients = classifier.coef_[0]
        indices = np.argsort(np.abs(coefficients))[-15:]

        print(f"\n{name} - Feature Coefficients (Top 15 by absolute value):")
        print("-" * 50)
        for idx in indices[::-1]:
```

```
            direction = "↑ increases risk" if coefficients[idx] > 0 else "↓␣
 ↪decreases risk"
            print(f"  {feat_names[idx]:<30}: {coefficients[idx]:+.6f}␣
 ↪({direction})")

        bar_colors = ['#404040' if c > 0 else '#a0a0a0' for c in␣
 ↪coefficients[indices]]
        ax.barh(range(len(indices)), coefficients[indices], color=bar_colors,
                edgecolor='black', linewidth=0.5)
        ax.set_yticks(range(len(indices)))
        ax.set_yticklabels([feat_names[i] for i in indices], fontsize=10)
        ax.legend(handles=[
            Patch(facecolor='#404040', edgecolor='black', label='Positive␣
 ↪(increases risk)'),
            Patch(facecolor='#a0a0a0', edgecolor='black', label='Negative␣
 ↪(decreases risk)')
        ], loc='lower right', fontsize=10, frameon=True)
        style_plot(ax, f"{name} - Feature Coefficients (Top 15)",
                   "Coefficient Value", "Features", legend=True, grid=False)

    ax.axvline(x=0, color='#cccccc', linestyle='-', linewidth=0.5)
    plt.tight_layout()
    plt.show()

# Re-fit on full training data for feature importance and SHAP
best_logreg.fit(X_train, y_train)
best_rf.fit(X_train, y_train)
best_xgb.fit(X_train, y_train)

feat_names = get_feature_names(X_train, numeric_features, categorical_features)

plot_feature_importance(best_logreg, "Logistic Regression", feat_names)
plot_feature_importance(best_rf,     "Random Forest",       feat_names)
plot_feature_importance(best_xgb,    "XGBoost",             feat_names)


#   Evaluation

logreg_cm, logreg_metrics = evaluate_model(y_test, logreg_test_prob,␣
 ↪logreg_threshold, "Logistic Regression")
rf_cm,     rf_metrics     = evaluate_model(y_test, rf_test_prob,     ␣
 ↪rf_threshold,     "Random Forest")
xgb_cm,    xgb_metrics    = evaluate_model(y_test, xgb_test_prob,    ␣
 ↪xgb_threshold,    "XGBoost")

comparison_df = pd.DataFrame({
```

```
    'Model':           ['Logistic Regression', 'Random Forest', 'XGBoost'],
    'Threshold':       [logreg_threshold, rf_threshold, xgb_threshold],
    'Sensitivity (%)': [m['sensitivity']*100 for m in [logreg_metrics,␣
 ↪rf_metrics, xgb_metrics]],
    'Specificity (%)': [m['specificity']*100 for m in [logreg_metrics,␣
 ↪rf_metrics, xgb_metrics]],
    'F1-Score':        [m['f1_score']         for m in [logreg_metrics,␣
 ↪rf_metrics, xgb_metrics]],
    'Precision (%)':   [m['precision']*100    for m in [logreg_metrics,␣
 ↪rf_metrics, xgb_metrics]],
    'Accuracy (%)':    [m['accuracy']*100     for m in [logreg_metrics,␣
 ↪rf_metrics, xgb_metrics]],
    'ROC-AUC':         [m['roc_auc']          for m in [logreg_metrics,␣
 ↪rf_metrics, xgb_metrics]],
}).round(2)

print("\nMODEL COMPARISON\n", comparison_df.to_string(index=False))


#   ROC Curves

colors = ['#1f77b4', '#ff7f0e', '#2ca02c']
fig, ax = plt.subplots(figsize=(8, 6))
for (name, prob), color in zip([("Logistic Regression", logreg_test_prob),
                                ("Random Forest",       rf_test_prob),
                                ("XGBoost",             xgb_test_prob)],␣
 ↪colors):
    fpr, tpr, _ = roc_curve(y_test, prob)
    ax.plot(fpr, tpr, color=color, linewidth=2.5,
            label=f"{name} (AUC={roc_auc_score(y_test, prob):.3f})", alpha=0.9)
ax.plot([0, 1], [0, 1], color='#cccccc', linestyle='--', linewidth=1.5,␣
 ↪label='Random (AUC=0.500)')
style_plot(ax, "ROC-AUC Curves", "False Positive Rate", "True Positive Rate")
ax.legend(fontsize=10, frameon=True, loc='lower right')
ax.set_xlim([-0.02, 1.02])
ax.set_ylim([-0.02, 1.02])
plt.tight_layout()
plt.show()


#   Confusion Matrices

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for cm, ax, metrics, name in zip(
    [logreg_cm, rf_cm, xgb_cm], axes,
    [logreg_metrics, rf_metrics, xgb_metrics],
```

```
    ['Logistic Regression', 'Random Forest', 'XGBoost']
):
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax,
                xticklabels=['Predicted Negative', 'Predicted Positive'],
                yticklabels=['Actual Negative', 'Actual Positive'],
                annot_kws={'size': 12, 'color': 'black', 'fontweight': 'bold'},
                cbar_kws={'label': 'Count', 'shrink': 0.8})
    ax.set_xlabel("Predicted Label", fontsize=12, labelpad=10, color='black')
    ax.set_ylabel("Actual Label",    fontsize=12, labelpad=10, color='black')
    ax.set_title(f"{name}\nSensitivity: {metrics['sensitivity']:.2%}  "
                 f"Specificity: {metrics['specificity']:.2%}",
                 fontsize=12, fontweight='bold', pad=15, color='black')
    ax.tick_params(axis='both', labelsize=10, colors='black')
plt.tight_layout()
plt.show()


#   Calibration Error Bar Chart

fig, ax = plt.subplots(figsize=(9, 6))

calibration_data = []
for prob, label in zip(
    [logreg_test_prob, rf_test_prob, xgb_test_prob],
    ["Logistic Regression", "Random Forest", "XGBoost"],
):
    prob_true, prob_pred = calibration_curve(y_test, prob, n_bins=10)
    calibration_data.append((prob_pred, prob_true, label))

bar_width = 0.25
index = np.arange(len(calibration_data[0][0]))
gray_shades = ['#404040', '#808080', '#b0b0b0']

for i, (prob_pred, prob_true, label) in enumerate(calibration_data):
    ax.bar(
        index + (i - 1) * bar_width,
        np.abs(prob_true - prob_pred),
        bar_width,
        label=label,
        color=gray_shades[i],
        edgecolor='black',
        linewidth=0.5,
    )

ax.set_xticks(index)
ax.set_xticklabels(
    [f'{x:.2f}' for x in calibration_data[0][0]],
```

```
    fontsize=10, rotation=45, ha='right'
)
style_plot(
    ax,
    "Calibration Error by Bin",
    "Probability Bins",
    "Absolute Calibration Error",
    legend=True,
    grid=True,
)
ax.legend(fontsize=10, frameon=True, loc='upper left', bbox_to_anchor=(1, 1))
plt.tight_layout()
plt.show()


#   SHAP Analysis (best model)

X_test_transformed_full = ColumnTransformer([
    ("num", MinMaxScaler(), numeric_features),
    ("cat", OneHotEncoder(drop="first", sparse_output=False),␣
 ↪categorical_features)
]).fit(X_train).transform(X_test)

best_raw_classifier = best_raw_pipeline.named_steps["classifier"]

if hasattr(best_raw_classifier, "feature_importances_"):
    explainer   = shap.TreeExplainer(best_raw_classifier)
    shap_values = explainer.shap_values(X_test_transformed_full)
    if isinstance(shap_values, list):
        shap_values_class1 = np.array(shap_values[1])
        expected_value     = float(np.array(explainer.expected_value).flat[1])
    else:
        shap_values_class1 = np.array(shap_values)
        expected_value     = float(np.array(explainer.expected_value).flat[0])
elif hasattr(best_raw_classifier, "coef_"):
    explainer   = shap.LinearExplainer(best_raw_classifier,␣
 ↪X_test_transformed_full)
    shap_values = explainer.shap_values(X_test_transformed_full)
    shap_values_class1 = np.array(shap_values)
    expected_value     = float(np.array(explainer.expected_value).flat[0])
else:
    raise ValueError(f"Unsupported classifier type:␣
 ↪{type(best_raw_classifier)}")

if shap_values_class1.ndim == 3:
    shap_values_class1 = shap_values_class1[:, :, 1]
shap_values_class1 = shap_values_class1.reshape(len(X_test), len(feat_names))
```

```
best_calibrated_prob_full = best_calibrated_model.predict_proba(X_test)[:, 1]

SHAP_SAMPLE_SIZE = min(4, len(X_test))
X_test_subset               = X_test.iloc[:SHAP_SAMPLE_SIZE]
y_test_subset               = y_test.iloc[:SHAP_SAMPLE_SIZE]
shap_values_subset          = shap_values_class1[:SHAP_SAMPLE_SIZE]
best_calibrated_prob_subset = best_calibrated_prob_full[:SHAP_SAMPLE_SIZE]


#   Waterfall Plots

def plot_grayscale_waterfall(shap_vals_row, expected_val, feat_names,
                             calibrated_prob, true_diagnosis, max_display=10,␣
 ↪patient_num=None):
    feature_impacts = [(name, float(val)) for name, val in zip(feat_names,␣
 ↪shap_vals_row)]
    feature_impacts.sort(key=lambda x: abs(x[1]), reverse=True)
    top = feature_impacts[:max_display]
    pos = sorted([(n, v) for n, v in top if v > 0], key=lambda x: x[1],␣
 ↪reverse=True)
    neg = sorted([(n, v) for n, v in top if v < 0], key=lambda x: x[1])
    sorted_features = pos + neg

    labels = [n for n, _ in sorted_features]
    values = [v for _, v in sorted_features]

    fig, ax = plt.subplots(figsize=(9, 6))
    ax.barh(
        range(len(values)), values,
        color=['#404040' if v > 0 else '#a0a0a0' for v in values],
        edgecolor='black', linewidth=0.5, height=0.7
    )

    # Padding based on value range so labels never touch the axis spine
    all_abs = [abs(v) for v in values if v != 0]
    pad = max(all_abs) * 0.08 if all_abs else 0.05

    for i, v in enumerate(values):
        label_text = f'+{v:.3f}' if v > 0 else f'{v:.3f}'
        bar_len = abs(v)
        # Inside bar when wide enough, outside otherwise
        if bar_len > pad * 2.5:
            x_pos  = v - pad if v > 0 else v + pad
            ha     = 'right' if v > 0 else 'left'
            color  = 'white'
        else:
```

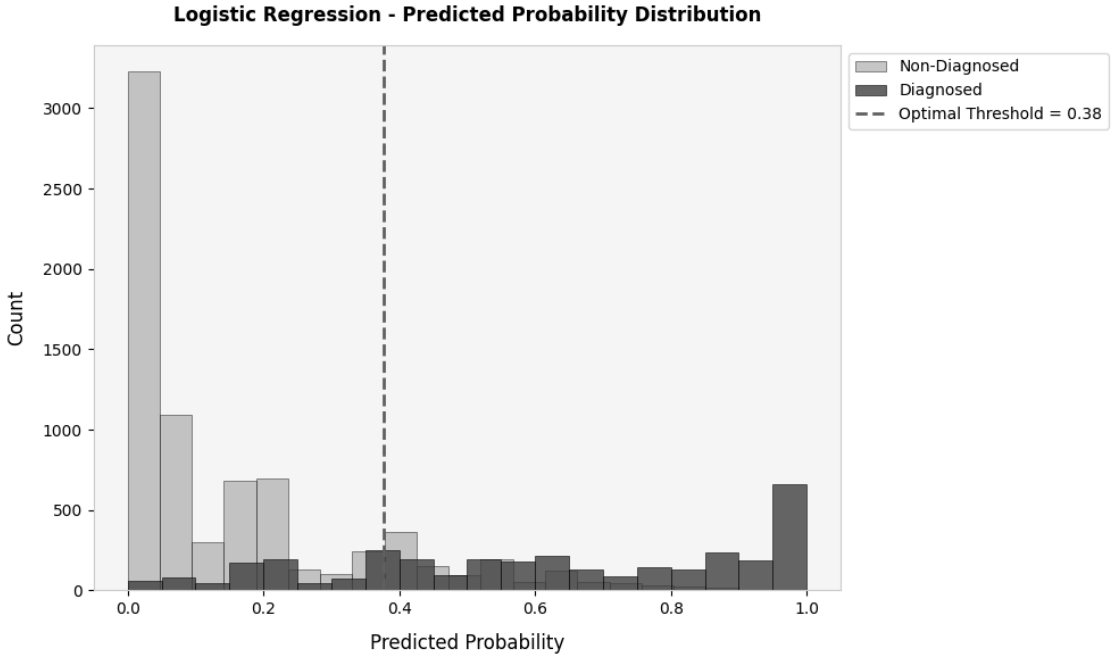
```
            x_pos  = v + pad if v > 0 else v - pad
            ha     = 'left' if v > 0 else 'right'
            color  = 'black'
        ax.text(x_pos, i, label_text, va='center', ha=ha,
                fontsize=9, color=color, fontweight='bold')

    # Expand x-axis so outside labels are never clipped
    x_min   = min(values + [0])
    x_max   = max(values + [0])
    x_range = x_max - x_min if x_max != x_min else 0.1
    ax.set_xlim(x_min - x_range * 0.22, x_max + x_range * 0.22)

    ax.set_yticks(range(len(labels)))
    ax.set_yticklabels(labels, fontsize=11)
    ax.axvline(x=expected_val, color='#606060', linestyle='--', linewidth=1.5,␣
↪alpha=0.7,
               label=f'Base value ({expected_val:.3f})')
    ax.axvline(x=0, color='#cccccc', linestyle='-', linewidth=0.5)

    diagnosis_text = 'DIAGNOSED' if true_diagnosis == 1 else 'NOT DIAGNOSED'
    ax.set_title(
        f"Patient {patient_num} ({best_model_name}) - {diagnosis_text}\n"
        f"Calibrated Probability: {calibrated_prob:.2%}  |  Base value:␣
↪{expected_val:.3f}",
        fontsize=12, fontweight='bold', pad=15, color='black'
    )
    ax.set_xlabel('SHAP Value (Log Odds)', fontsize=12, labelpad=10,␣
↪color='black')
    ax.set_ylabel('Features',              fontsize=12, labelpad=10,␣
↪color='black')
    ax.grid(False)
    ax.tick_params(axis='both', labelsize=10, colors='black')
    ax.set_facecolor('#f5f5f5')
    for spine in ax.spines.values():
        spine.set_color('#cccccc')
    plt.tight_layout()
    plt.show()

for i in range(len(X_test_subset)):
    plot_grayscale_waterfall(
        np.array(shap_values_subset[i], dtype=float).flatten(),
        expected_value, feat_names,
        best_calibrated_prob_subset[i],
        y_test_subset.iloc[i],
        max_display=10, patient_num=i + 1
    )
```

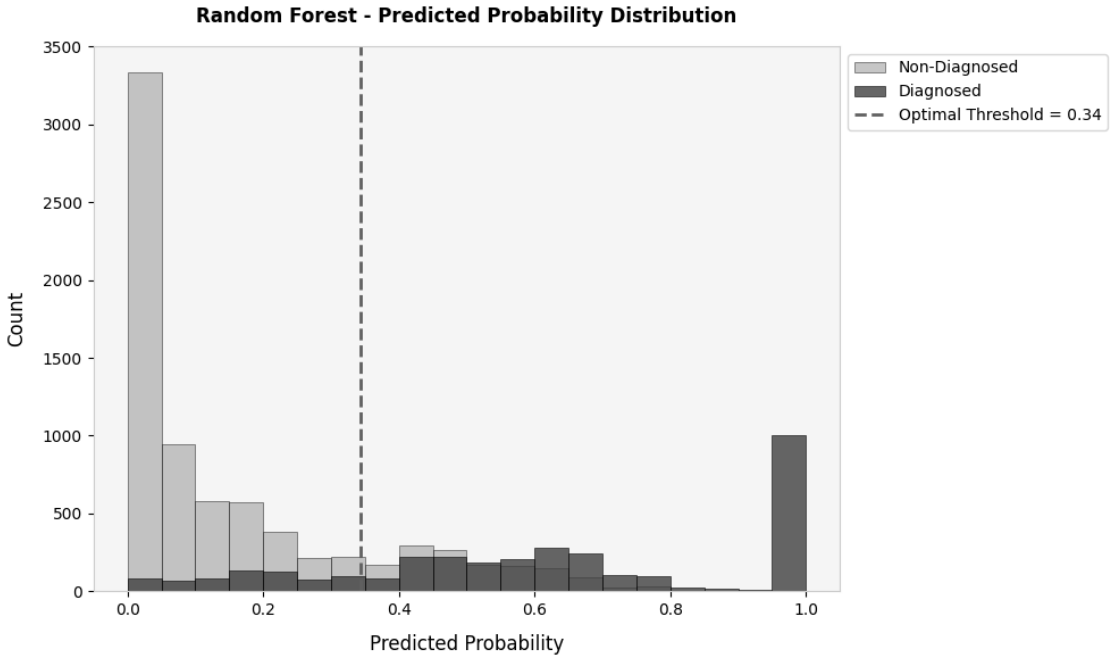
```
#   Results Table (full dataset)

top_increase = [get_top_shap_features(shap_values_class1[i], feat_names,␣
 ↪positive=True)
                for i in range(len(best_calibrated_prob_full))]
top_decrease = [get_top_shap_features(shap_values_class1[i], feat_names,␣
 ↪positive=False)
                for i in range(len(best_calibrated_prob_full))]

results_table = pd.DataFrame({
    "Patient":            range(1, len(best_calibrated_prob_full) + 1),
    "Diagnosis":          ["Diagnosed" if v == 1 else "Not Diagnosed" for v in␣
 ↪y_test.values],
    "Predicted Prob (%)": np.round(best_calibrated_prob_full * 100, 2),
    "Risk Category":      [categorize_risk(p) for p in␣
 ↪best_calibrated_prob_full],
    "Top 5 Increasing":   [", ".join(f for f in feat if f) for feat in␣
 ↪top_increase],
    "Top 5 Decreasing":   [", ".join(f for f in feat if f) for feat in␣
 ↪top_decrease],
})

results_table.to_csv("predictions_with_explanation.csv", index=False)

def print_patient_report(rows, title):
    print(f"\n{title}")
    print("-" * 110)
    for _, row in rows.iterrows():
        print(f"Patient: {row['Patient']:<5} | Diagnosis: {row['Diagnosis']:
 ↪<13} | "
              f"Prob: {row['Predicted Prob (%)']:<6}% | Risk: {row['Risk␣
 ↪Category']:<12}")
        print(f"  ↑ Increasing: {row['Top 5 Increasing']}")
        print(f"  ↓ Decreasing: {row['Top 5 Decreasing']}")
        print("-" * 110)

print_patient_report(results_table.head(10), f"FIRST 10 PATIENTS -␣
 ↪{best_model_name}")
print_patient_report(results_table.tail(10), f"LAST 10 PATIENTS  -␣
 ↪{best_model_name}")


#   Risk Distribution (full test set)

best_test_prob_full = best_calibrated_model.predict_proba(X_test)[:, 1]
```

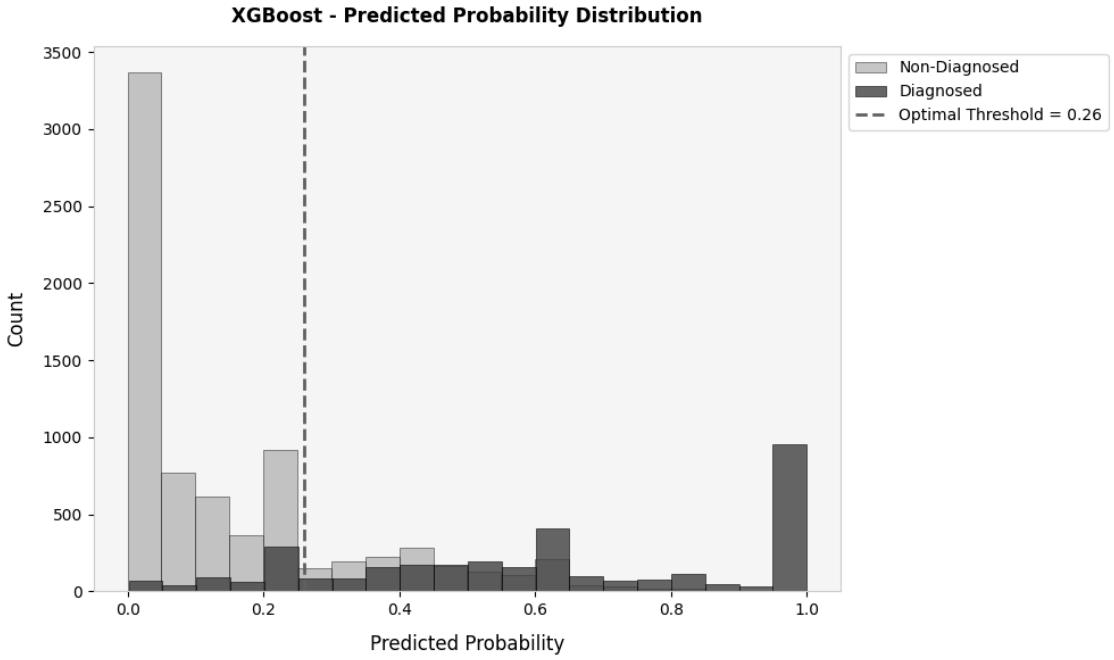
```
risk_summary = (
    pd.Series([categorize_risk(p) for p in best_test_prob_full], name='Risk␣
 ↪Category')
    .value_counts()
    .reindex(["Low Risk", "Moderate Risk", "High Risk"])
    .dropna()
    .to_frame('Count')
)
risk_summary['Percentage'] = (risk_summary['Count'] / len(best_test_prob_full)␣
 ↪* 100).round(2)
print(f"\nRISK DISTRIBUTION - {best_model_name} (n={len(best_test_prob_full)})")
print(risk_summary.to_string())


#   Save Best Model

model_filename = f"predictive_model_{best_model_name.lower().replace(' ', '_')}.
 ↪pkl"
joblib.dump(best_calibrated_model, model_filename, compress=3)
```

```
PREDICTED PROBABILITIES - First 20 Patients
 Patient  Diagnosis (1=Anxiety, 0=No)  Logistic Regression (%)  Random Forest
(%)  XGBoost (%)
       1                            1                    81.56
72.52         69.38
       2                            0                     0.25
0.56          0.17
       3                            0                     0.44
6.66          0.17
       4                            0                     6.21
8.65         11.31
       5                            0                     0.00
0.00          0.00
       6                            0                    55.64
59.92         60.57
       7                            0                    21.18
37.76         24.89
       8                            0                    15.98
18.06         15.15
       9                            0                     1.55
5.29          1.14
      10                            0                    18.00
11.55         11.42
      11                            0                     0.56
3.18          3.16
      12                            0                     0.25
```

```
0.00          0.17
      13                          0                      41.45
44.89         43.70
      14                          0                      38.76
38.86         24.89
      15                          0                      11.81
7.73          13.10
      16                          0                       0.56
0.00          0.47
      17                          0                       8.65
0.99          6.35
      18                          0                      36.13
34.74         24.89
      19                          0                       9.07
11.38         18.07
      20                          1                      98.72
100.00        100.00
```

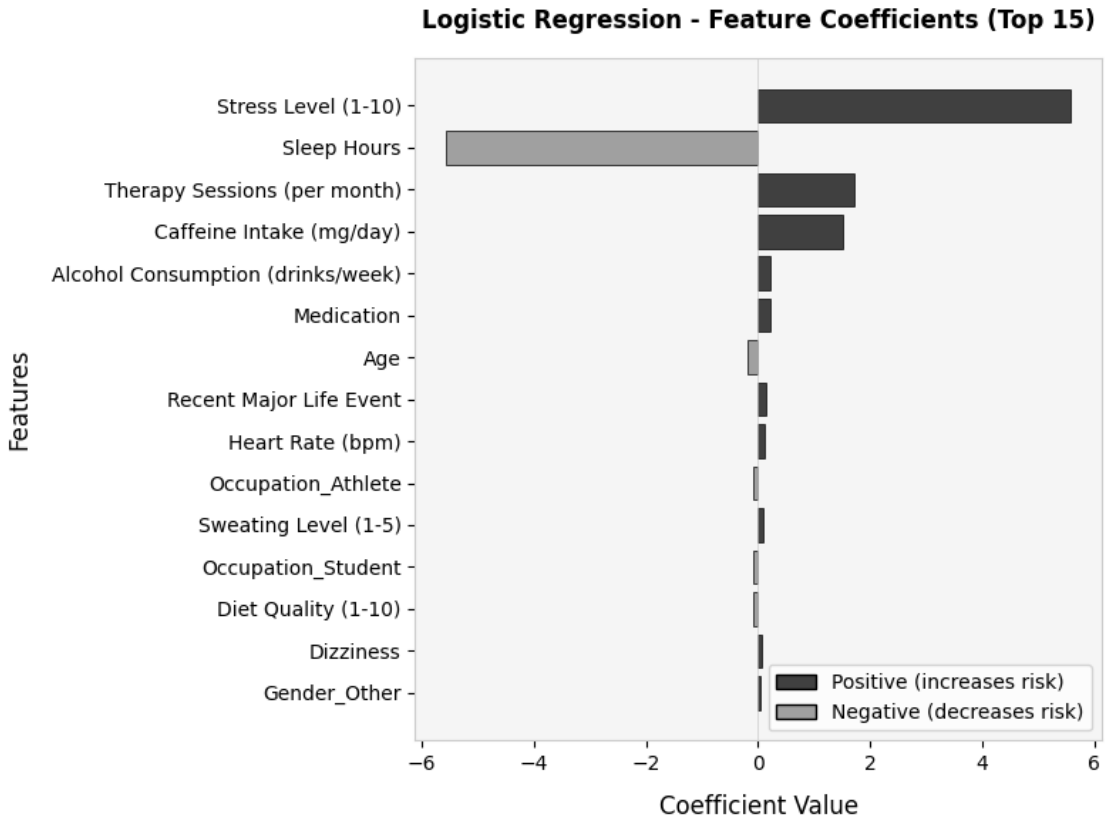






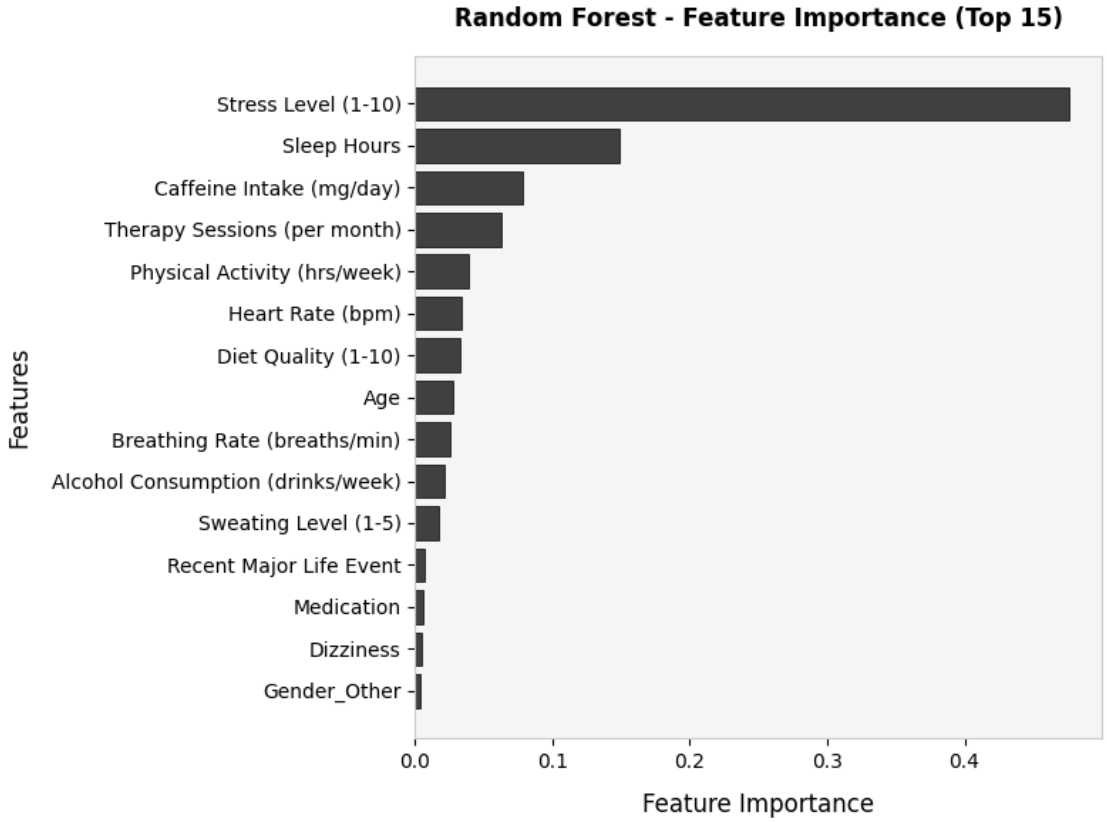

```
Logistic Regression - Feature Coefficients (Top 15 by absolute value):
------------------------------------------------
  Stress Level (1-10)           : +5.579628 (↑ increases risk)
```

```
  Sleep Hours                    : -5.567246 (↓ decreases risk)
  Therapy Sessions (per month)   : +1.727037 (↑ increases risk)
  Caffeine Intake (mg/day)       : +1.517290 (↑ increases risk)
  Alcohol Consumption (drinks/week): +0.225831 (↑ increases risk)
  Medication                     : +0.210463 (↑ increases risk)
  Age                            : -0.190901 (↓ decreases risk)
  Recent Major Life Event        : +0.143319 (↑ increases risk)
  Heart Rate (bpm)               : +0.122618 (↑ increases risk)
  Occupation_Athlete             : -0.097862 (↓ decreases risk)
  Sweating Level (1-5)           : +0.092717 (↑ increases risk)
  Occupation_Student             : -0.089914 (↓ decreases risk)
  Diet Quality (1-10)            : -0.088284 (↓ decreases risk)
  Dizziness                      : +0.061192 (↑ increases risk)
  Gender_Other                   : +0.051069 (↑ increases risk)
```



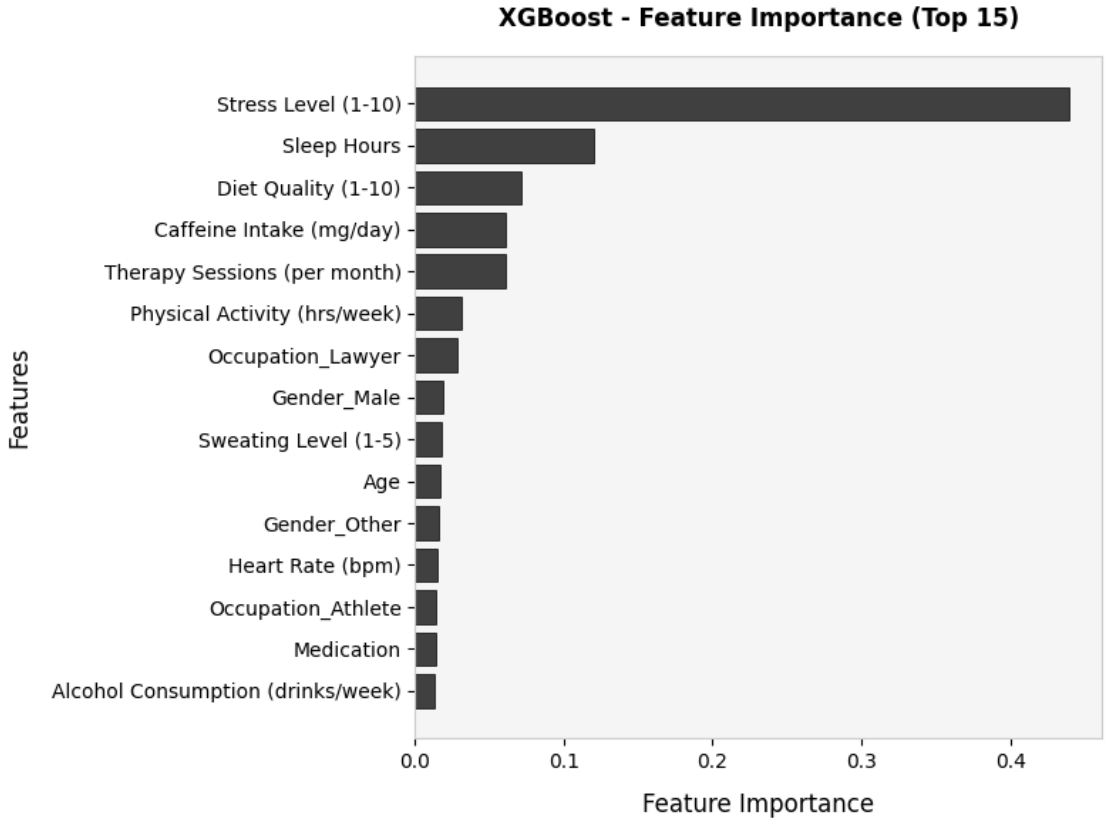

```
Random Forest - Feature Importances (Top 15):
--------------------------------------------------
  Stress Level (1-10)            : 0.476110
  Sleep Hours                    : 0.148280
  Caffeine Intake (mg/day)       : 0.078679
```

```
  Therapy Sessions (per month)  : 0.062334
  Physical Activity (hrs/week)  : 0.039263
  Heart Rate (bpm)              : 0.033535
  Diet Quality (1-10)           : 0.032816
  Age                           : 0.027579
  Breathing Rate (breaths/min)  : 0.025976
  Alcohol Consumption (drinks/week): 0.021759
  Sweating Level (1-5)          : 0.016825
  Recent Major Life Event       : 0.006445
  Medication                    : 0.005368
  Dizziness                     : 0.004420
  Gender_Other                  : 0.003558
```



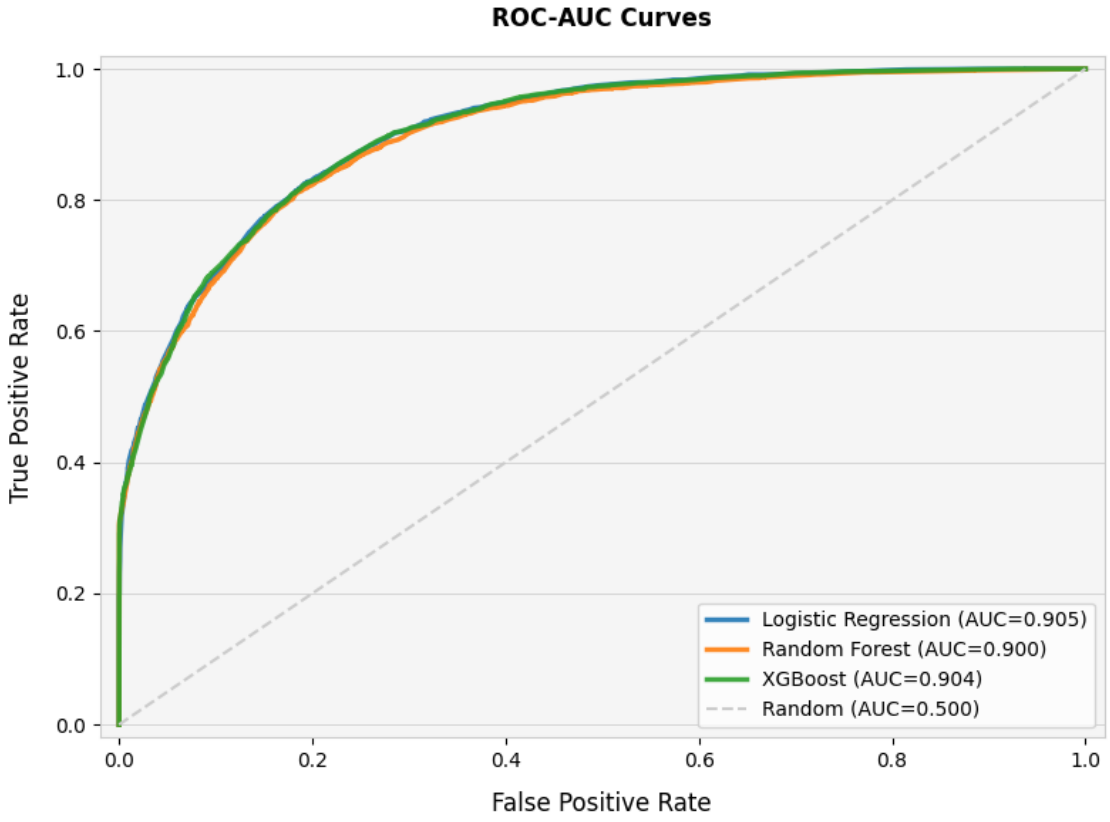

```
XGBoost - Feature Importances (Top 15):
------------------------------------------------
  Stress Level (1-10)           : 0.439649
  Sleep Hours                   : 0.120012
  Diet Quality (1-10)           : 0.071663
  Caffeine Intake (mg/day)      : 0.060706
  Therapy Sessions (per month)  : 0.060516
```

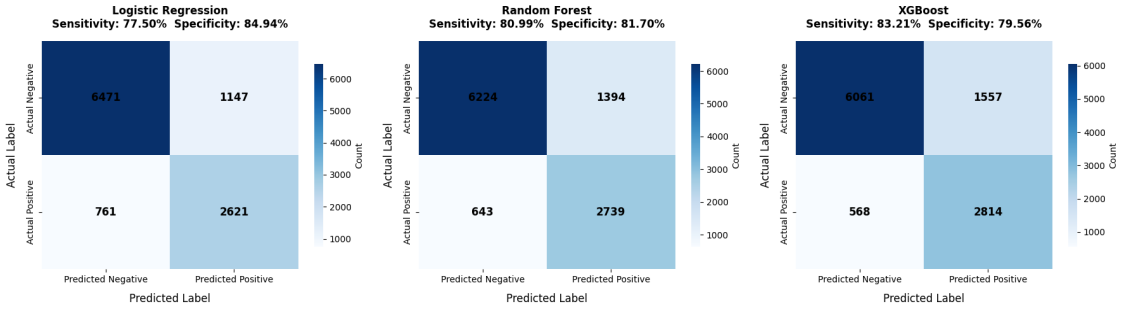
```
  Physical Activity (hrs/week)  : 0.030854
  Occupation_Lawyer             : 0.028563
  Gender_Male                   : 0.019071
  Sweating Level (1-5)          : 0.018077
  Age                           : 0.017322
  Gender_Other                  : 0.016230
  Heart Rate (bpm)              : 0.015048
  Occupation_Athlete            : 0.014324
  Medication                    : 0.013791
  Alcohol Consumption (drinks/week): 0.013408
```

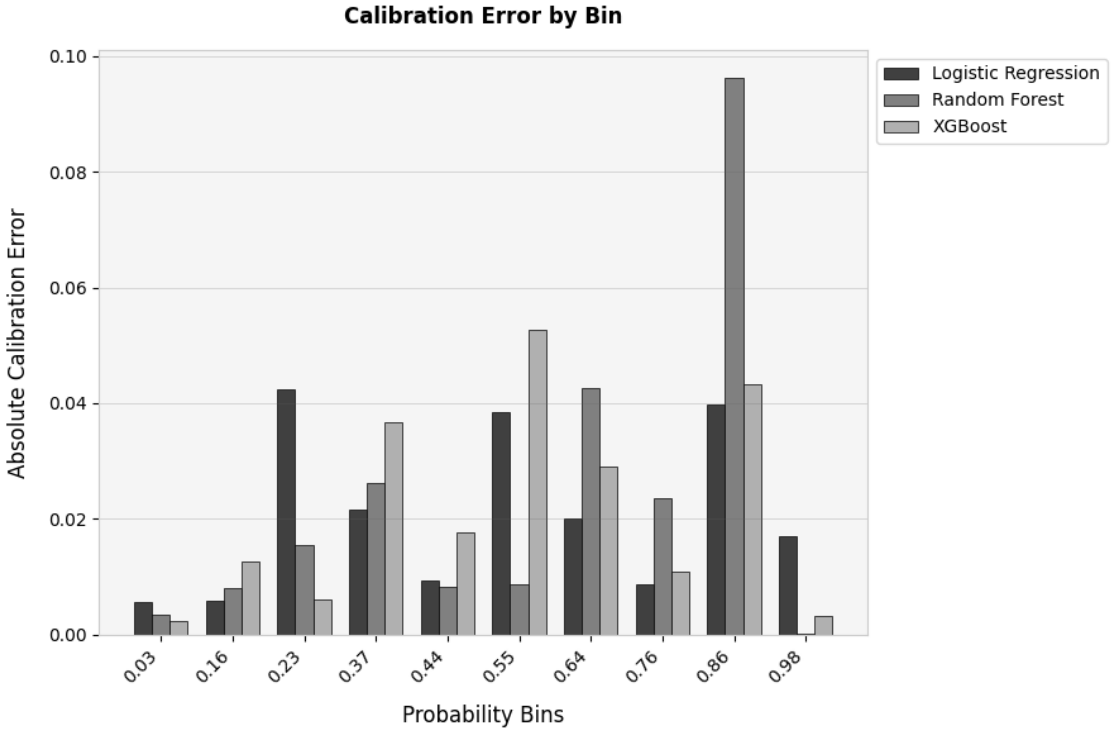


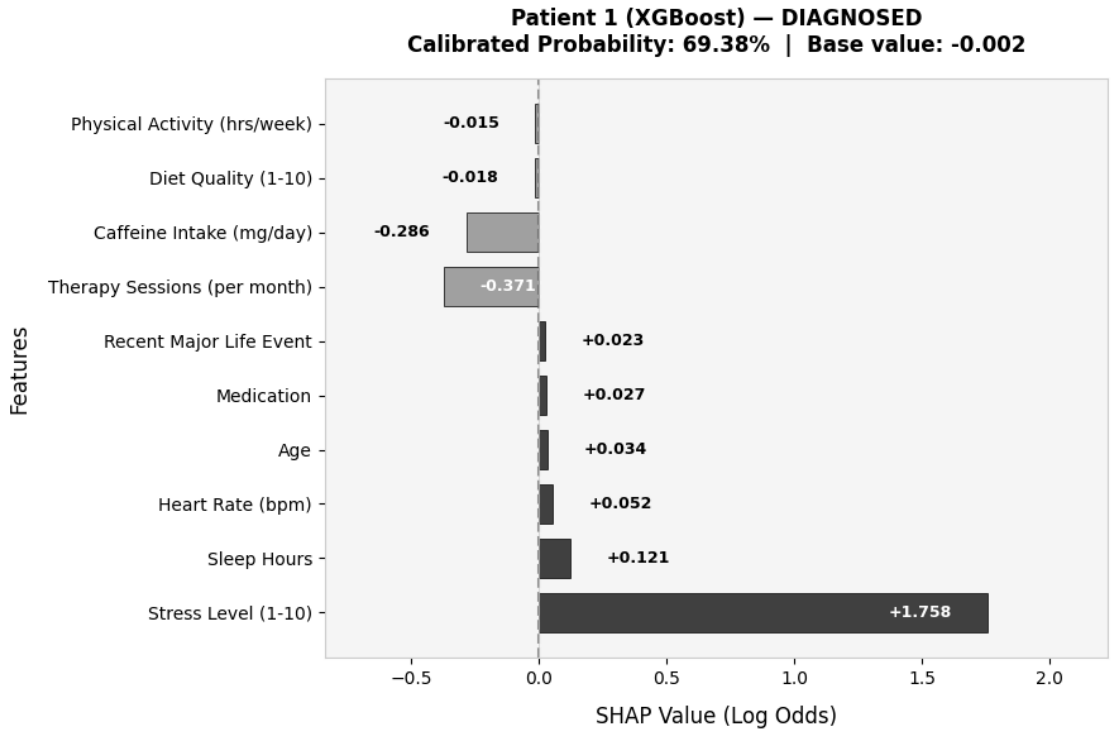

```
Logistic Regression (threshold=0.377)
  TN=6471  FP=1147  FN=761  TP=2621
  Sensitivity: 77.50%  Specificity: 84.94%
  F1: 0.7331  Accuracy: 82.65%  Precision: 69.56%
  ROC-AUC: 0.9049  Brier: 0.1114
              precision    recall  f1-score   support

           0     0.8948    0.8494    0.8715      7618
           1     0.6956    0.7750    0.7331      3382
```

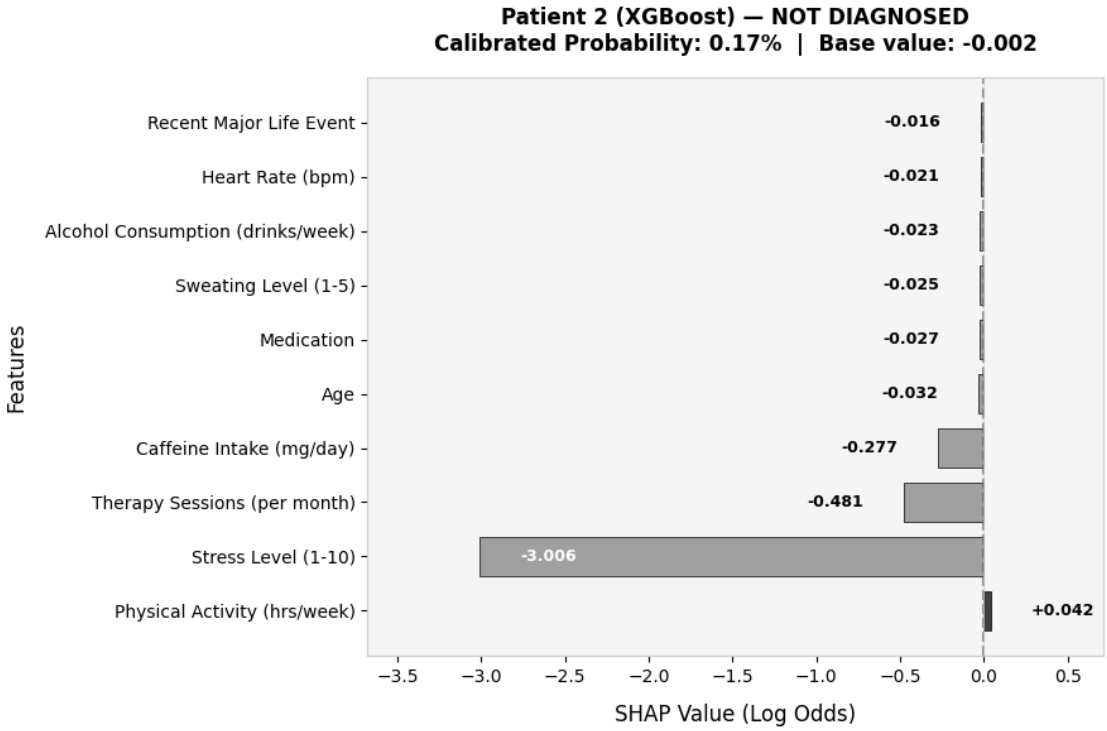
```
    accuracy                           0.8265     11000
   macro avg     0.7952    0.8122    0.8023     11000
weighted avg     0.8335    0.8265    0.8290     11000


Random Forest (threshold=0.343)
  TN=6224  FP=1394  FN=643  TP=2739
  Sensitivity: 80.99%  Specificity: 81.70%
  F1: 0.7289  Accuracy: 81.48%  Precision: 66.27%
  ROC-AUC: 0.8997  Brier: 0.1131
              precision    recall  f1-score   support

           0     0.9064    0.8170    0.8594      7618
           1     0.6627    0.8099    0.7289      3382

    accuracy                           0.8148     11000
   macro avg     0.7845    0.8134    0.7942     11000
weighted avg     0.8315    0.8148    0.8193     11000


XGBoost (threshold=0.260)
  TN=6061  FP=1557  FN=568  TP=2814
  Sensitivity: 83.21%  Specificity: 79.56%
  F1: 0.7259  Accuracy: 80.68%  Precision: 64.38%
  ROC-AUC: 0.9041  Brier: 0.1116
              precision    recall  f1-score   support

           0     0.9143    0.7956    0.8508      7618
           1     0.6438    0.8321    0.7259      3382

    accuracy                           0.8068     11000
   macro avg     0.7791    0.8138    0.7884     11000
weighted avg     0.8311    0.8068    0.8124     11000


MODEL COMPARISON
               Model  Threshold  Sensitivity (%)  Specificity (%)  F1-Score
Precision (%)  Accuracy (%)  ROC-AUC
Logistic Regression       0.38            77.50            84.94      0.73
69.56         82.65      0.9
      Random Forest       0.34            80.99            81.70      0.73
66.27         81.48      0.9
            XGBoost       0.26            83.21            79.56      0.73
64.38         80.68      0.9
```

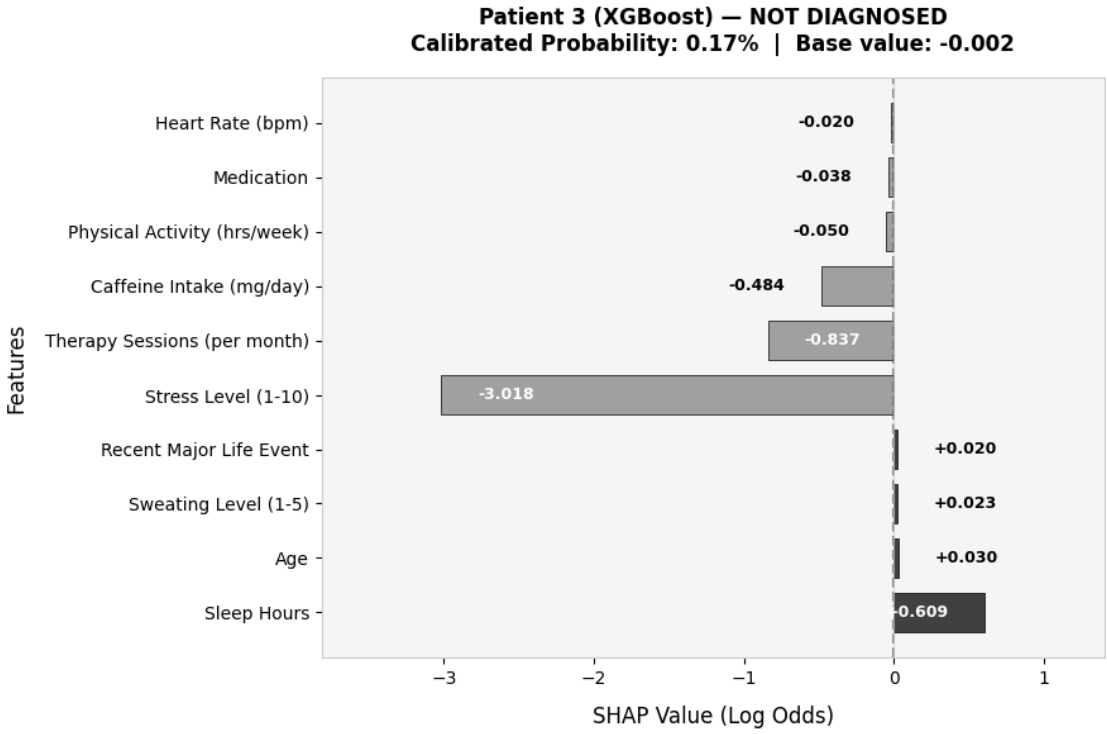

ROC-AUC Curves
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Logistic Regression (AUC=0.905)
Random Forest (AUC=0.900)
XGBoost (AUC=0.904)
Random (AUC=0.500)

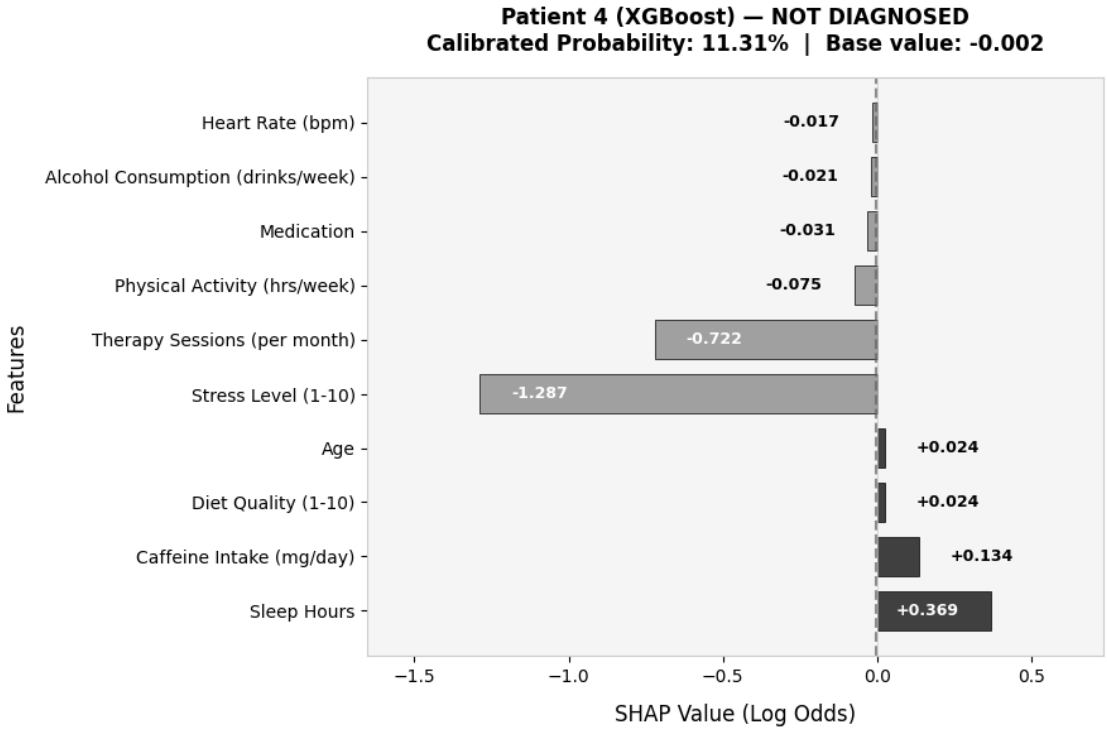

Logistic Regression
Sensitivity: 77.50% Specificity: 84.94%
6471
1147
761
2621
Actual Label
Actual Negative
Actual Positive
Predicted Negative
Predicted Positive
Predicted Label
Count
Random Forest
Sensitivity: 80.99% Specificity: 81.70%
6224
1394
643
2739
XGBoost
Sensitivity: 83.21% Specificity: 79.56%
6061
1557
568
2814


Calibration Error by Bin
Logistic Regression
Random Forest
XGBoost
Absolute Calibration Error
0.10
0.08
0.06
0.04
0.02
0.00
0.03
0.16
0.23
0.37
0.44
0.55
0.64
0.76
0.86
0.98
Probability Bins


Patient 1 (XGBoost) — DIAGNOSED
Calibrated Probability: 69.38% | Base value: -0.002
Features
Physical Activity (hrs/week)
Diet Quality (1-10)
Caffeine Intake (mg/day)
Therapy Sessions (per month)
Recent Major Life Event
Medication
Age
Heart Rate (bpm)
Sleep Hours
Stress Level (1-10)
-0.015
-0.018
-0.286
-0.371
+0.023
+0.027
+0.034
+0.052
+0.121
+1.758
−0.5
0.0
0.5
1.0
1.5
2.0
SHAP Value (Log Odds)


Patient 2 (XGBoost) — NOT DIAGNOSED
Calibrated Probability: 0.17% | Base value: -0.002
Features
Recent Major Life Event
Heart Rate (bpm)
Alcohol Consumption (drinks/week)
Sweating Level (1-5)
Medication
Age
Caffeine Intake (mg/day)
Therapy Sessions (per month)
Stress Level (1-10)
Physical Activity (hrs/week)
-0.016
-0.021
-0.023
-0.025
-0.027
-0.032
-0.277
-0.481
-3.006
+0.042
−3.5
−3.0
−2.5
−2.0
−1.5
−1.0
−0.5
0.0
0.5
SHAP Value (Log Odds)


Patient 3 (XGBoost) — NOT DIAGNOSED
Calibrated Probability: 0.17% | Base value: -0.002
Features
Heart Rate (bpm)
Medication
Physical Activity (hrs/week)
Caffeine Intake (mg/day)
Therapy Sessions (per month)
Stress Level (1-10)
Recent Major Life Event
Sweating Level (1-5)
Age
Sleep Hours
-0.020
-0.038
-0.050
-0.484
-0.837
-3.018
+0.020
+0.023
+0.030
+0.609
−3
−2
−1
0
1
SHAP Value (Log Odds)




```
FIRST 10 PATIENTS - XGBoost
--------------------------------------------------------------------------------
------------------------------
Patient: 1     | Diagnosis: Diagnosed     | Prob: 69.38 % | Risk: Moderate Risk
  ↑ Increasing: Stress Level (1-10) (+1.758), Sleep Hours (+0.121), Heart Rate
(bpm) (+0.052), Age (+0.034), Medication (+0.027)
  ↓ Decreasing: Therapy Sessions (per month) (-0.371), Caffeine Intake (mg/day)
(-0.286), Diet Quality (1-10) (-0.018), Physical Activity (hrs/week) (-0.015),
Breathing Rate (breaths/min) (-0.011)
--------------------------------------------------------------------------------
------------------------------
Patient: 2     | Diagnosis: Not Diagnosed | Prob: 0.17  % | Risk: Low Risk
  ↑ Increasing: Physical Activity (hrs/week) (+0.042), Sleep Hours (+0.011),
Occupation_Athlete (+0.002), Breathing Rate (breaths/min) (+0.002), Gender_Other
(+0.001)
  ↓ Decreasing: Stress Level (1-10) (-3.006), Therapy Sessions (per month)
(-0.481), Caffeine Intake (mg/day) (-0.277), Age (-0.032), Medication (-0.027)
--------------------------------------------------------------------------------
------------------------------
Patient: 3     | Diagnosis: Not Diagnosed | Prob: 0.17  % | Risk: Low Risk
  ↑ Increasing: Sleep Hours (+0.609), Age (+0.030), Sweating Level (1-5)
(+0.023), Recent Major Life Event (+0.020), Occupation_Other (+0.008)
  ↓ Decreasing: Stress Level (1-10) (-3.018), Therapy Sessions (per month)
```

(-0.837), Caffeine Intake (mg/day) (-0.484), Physical Activity (hrs/week) (-0.050), Medication (-0.038)

Patient: 4 | Diagnosis: Not Diagnosed | Prob: 11.31 % | Risk: Low Risk
↑ Increasing: Sleep Hours (+0.369), Caffeine Intake (mg/day) (+0.134), Diet Quality (1-10) (+0.024), Age (+0.024), Sweating Level (1-5) (+0.015)
↓ Decreasing: Stress Level (1-10) (-1.287), Therapy Sessions (per month) (-0.722), Physical Activity (hrs/week) (-0.075), Medication (-0.031), Alcohol Consumption (drinks/week) (-0.021)

Patient: 5 | Diagnosis: Not Diagnosed | Prob: 0.0 % | Risk: Low Risk
↑ Increasing: Medication (+0.036), Occupation_Other (+0.029), Diet Quality (1-10) (+0.016), Smoking (+0.009), Sweating Level (1-5) (+0.005)
↓ Decreasing: Stress Level (1-10) (-2.987), Therapy Sessions (per month) (-0.655), Sleep Hours (-0.387), Caffeine Intake (mg/day) (-0.208), Age (-0.054)

Patient: 6 | Diagnosis: Not Diagnosed | Prob: 60.57 % | Risk: Moderate Risk
↑ Increasing: Stress Level (1-10) (+1.484), Medication (+0.038), Physical Activity (hrs/week) (+0.027), Recent Major Life Event (+0.015), Sweating Level (1-5) (+0.015)
↓ Decreasing: Therapy Sessions (per month) (-0.438), Caffeine Intake (mg/day) (-0.104), Alcohol Consumption (drinks/week) (-0.059), Age (-0.046), Diet Quality (1-10) (-0.025)

Patient: 7 | Diagnosis: Not Diagnosed | Prob: 24.89 % | Risk: Low Risk
↑ Increasing: Stress Level (1-10) (+0.842), Heart Rate (bpm) (+0.058), Age (+0.039), Recent Major Life Event (+0.026), Alcohol Consumption (drinks/week) (+0.023)
↓ Decreasing: Therapy Sessions (per month) (-0.686), Caffeine Intake (mg/day) (-0.640), Sleep Hours (-0.446), Medication (-0.041), Gender_Male (-0.012)

Patient: 8 | Diagnosis: Not Diagnosed | Prob: 15.15 % | Risk: Low Risk
↑ Increasing: Caffeine Intake (mg/day) (+0.495), Alcohol Consumption (drinks/week) (+0.058), Physical Activity (hrs/week) (+0.030), Heart Rate (bpm) (+0.018), Breathing Rate (breaths/min) (+0.010)
↓ Decreasing: Stress Level (1-10) (-0.894), Therapy Sessions (per month) (-0.641), Age (-0.043), Diet Quality (1-10) (-0.039), Recent Major Life Event (-0.029)

Patient: 9 | Diagnosis: Not Diagnosed | Prob: 1.14 % | Risk: Low Risk
↑ Increasing: Caffeine Intake (mg/day) (+0.343), Recent Major Life Event (+0.016), Occupation_Lawyer (+0.008), Breathing Rate (breaths/min) (+0.005),

```
Occupation_Athlete (+0.002)
  ↓ Decreasing: Stress Level (1-10) (-2.710), Therapy Sessions (per month)
(-0.448), Alcohol Consumption (drinks/week) (-0.092), Physical Activity
(hrs/week) (-0.042), Sleep Hours (-0.037)
------------------------------------------------------------------------------
------------------------------
Patient: 10    | Diagnosis: Not Diagnosed | Prob: 11.42 % | Risk: Low Risk
  ↑ Increasing: Sleep Hours (+0.668), Medication (+0.042), Alcohol Consumption
(drinks/week) (+0.036), Recent Major Life Event (+0.022), Sweating Level (1-5)
(+0.016)
  ↓ Decreasing: Stress Level (1-10) (-0.900), Therapy Sessions (per month)
(-0.836), Caffeine Intake (mg/day) (-0.470), Age (-0.042), Physical Activity
(hrs/week) (-0.019)
------------------------------------------------------------------------------
------------------------------

LAST 10 PATIENTS  - XGBoost
------------------------------------------------------------------------------
------------------------------
Patient: 10991 | Diagnosis: Not Diagnosed | Prob: 0.0   % | Risk: Low Risk
  ↑ Increasing: Medication (+0.031), Alcohol Consumption (drinks/week) (+0.030),
Age (+0.023), Occupation_Athlete (+0.003), Occupation_Scientist (+0.001)
  ↓ Decreasing: Stress Level (1-10) (-2.715), Therapy Sessions (per month)
(-0.686), Sleep Hours (-0.410), Caffeine Intake (mg/day) (-0.199), Recent Major
Life Event (-0.025)
------------------------------------------------------------------------------
------------------------------
Patient: 10992 | Diagnosis: Not Diagnosed | Prob: 1.44  % | Risk: Low Risk
  ↑ Increasing: Caffeine Intake (mg/day) (+0.452), Alcohol Consumption
(drinks/week) (+0.071), Sweating Level (1-5) (+0.015), Diet Quality (1-10)
(+0.014), Occupation_Athlete (+0.002)
  ↓ Decreasing: Stress Level (1-10) (-2.759), Therapy Sessions (per month)
(-0.472), Sleep Hours (-0.077), Physical Activity (hrs/week) (-0.043), Recent
Major Life Event (-0.026)
------------------------------------------------------------------------------
------------------------------
Patient: 10993 | Diagnosis: Not Diagnosed | Prob: 35.73 % | Risk: Moderate Risk
  ↑ Increasing: Stress Level (1-10) (+0.288), Caffeine Intake (mg/day) (+0.063),
Alcohol Consumption (drinks/week) (+0.050), Medication (+0.032), Diet Quality
(1-10) (+0.030)
  ↓ Decreasing: Therapy Sessions (per month) (-0.380), Physical Activity
(hrs/week) (-0.060), Sleep Hours (-0.022), Age (-0.022), Heart Rate (bpm)
(-0.016)
------------------------------------------------------------------------------
------------------------------
Patient: 10994 | Diagnosis: Not Diagnosed | Prob: 0.17  % | Risk: Low Risk
  ↑ Increasing: Alcohol Consumption (drinks/week) (+0.033), Medication (+0.022),
Diet Quality (1-10) (+0.019), Sweating Level (1-5) (+0.005), Occupation_Athlete
```

(+0.002)
  ↓ Decreasing: Stress Level (1-10) (-2.865), Sleep Hours (-0.532), Therapy Sessions (per month) (-0.156), Caffeine Intake (mg/day) (-0.148), Age (-0.032)

Patient: 10995 | Diagnosis: Not Diagnosed | Prob: 15.46 % | Risk: Low Risk
  ↑ Increasing: Therapy Sessions (per month) (+0.551), Sleep Hours (+0.074), Medication (+0.028), Diet Quality (1-10) (+0.027), Alcohol Consumption (drinks/week) (+0.021)
  ↓ Decreasing: Stress Level (1-10) (-1.260), Caffeine Intake (mg/day) (-0.363), Age (-0.024), Recent Major Life Event (-0.022), Physical Activity (hrs/week) (-0.008)

Patient: 10996 | Diagnosis: Diagnosed | Prob: 56.67 % | Risk: Moderate Risk
  ↑ Increasing: Stress Level (1-10) (+0.943), Caffeine Intake (mg/day) (+0.475), Sleep Hours (+0.139), Medication (+0.048), Age (+0.034)
  ↓ Decreasing: Therapy Sessions (per month) (-0.669), Physical Activity (hrs/week) (-0.063), Sweating Level (1-5) (-0.029), Recent Major Life Event (-0.011), Heart Rate (bpm) (-0.008)

Patient: 10997 | Diagnosis: Not Diagnosed | Prob: 20.39 % | Risk: Low Risk
  ↑ Increasing: Stress Level (1-10) (+0.192), Alcohol Consumption (drinks/week) (+0.031), Sweating Level (1-5) (+0.016), Gender_Male (+0.006), Sleep Hours (+0.003)
  ↓ Decreasing: Caffeine Intake (mg/day) (-0.553), Therapy Sessions (per month) (-0.380), Recent Major Life Event (-0.032), Medication (-0.025), Age (-0.021)

Patient: 10998 | Diagnosis: Not Diagnosed | Prob: 32.41 % | Risk: Moderate Risk
  ↑ Increasing: Stress Level (1-10) (+0.853), Physical Activity (hrs/week) (+0.042), Age (+0.035), Alcohol Consumption (drinks/week) (+0.034), Medication (+0.026)
  ↓ Decreasing: Therapy Sessions (per month) (-0.454), Caffeine Intake (mg/day) (-0.419), Sleep Hours (-0.102), Sweating Level (1-5) (-0.029), Heart Rate (bpm) (-0.025)

Patient: 10999 | Diagnosis: Not Diagnosed | Prob: 5.6   % | Risk: Low Risk
  ↑ Increasing: Sleep Hours (+0.401), Medication (+0.036), Sweating Level (1-5) (+0.016), Diet Quality (1-10) (+0.015), Recent Major Life Event (+0.015)
  ↓ Decreasing: Stress Level (1-10) (-1.320), Therapy Sessions (per month) (-0.811), Caffeine Intake (mg/day) (-0.221), Age (-0.030), Physical Activity (hrs/week) (-0.029)

Patient: 11000 | Diagnosis: Not Diagnosed | Prob: 0.17 % | Risk: Low Risk

```
  ↑ Increasing: Sleep Hours (+0.113), Age (+0.031), Recent Major Life Event
(+0.031), Alcohol Consumption (drinks/week) (+0.024), Sweating Level (1-5)
(+0.011)
  ↓ Decreasing: Stress Level (1-10) (-2.965), Therapy Sessions (per month)
(-0.485), Caffeine Intake (mg/day) (-0.286), Diet Quality (1-10) (-0.019), Heart
Rate (bpm) (-0.018)
--------------------------------------------------------------------------------
------------------------------

RISK DISTRIBUTION - XGBoost (n=11000)
               Count  Percentage
Risk Category
Low Risk        6807       61.88
Moderate Risk   2822       25.65
High Risk       1371       12.46
```

[1]: ['predictive_model_xgboost.pkl']